README_Avance 2

Sistema de gestión de Transporte y logística **FleetLogix**

Análisis y optimización SQL

1. Descripcion del Avance

El Avance 2 ejecuta 12 queries SQL sobre la base de datos FleetLogix (505,556 registros), mide su performance con EXPLAIN ANALYZE, aplica 5 índices de optimización y documenta la mejora. La query más avanzada (C1) implementa un modelo de ranking bayesiano de eficiencia de conductores que pondera tres factores de negocio: tasa de entrega exitosa, puntualidad y eficiencia de combustible.

---

2. Las 12 Queries SQL

2.1 Nivel básico

#	Nombre	Pregunta de negocio	Conceptos SQL
B1	Vehículos y ultimo mantenimiento	¿Qué vehículos están en riesgo de falla?	LEFT JOIN, MAX(), GROUP BY, NULLS FIRST
B2	Rutas más activas	¿Qué rutas concentran más operación y combustible?	INNER JOIN, WHERE, COUNT, SUM, AVG, ROUND
B3	KPI estados de entrega	¿Qué % de entregas se completan exitosamente?	GROUP BY, OVER() para porcentaje sobre total

2.2 Nivel Intermedio

#	Nombre	Pregunta de negocio	Conceptos SQL
I1	Top conductores productivos	¿Quiénes son los conductores más productivos?	JOIN 3 tablas, COUNT DISTINCT, HAVING, INTERVAL
I2	Eficiencia por tipo vehículo	¿Qué tipo de vehículo tiene menor consumo?	LEFT JOINs múltiples, condición en ON, NULLIF
I3	Reporte mensual operaciones	¿Como evoluciona la operación mes a mes?	EXTRACT, CASE WHEN, conteo condicional
I4	Tiempo real vs estimado	¿Qué rutas se desvían del tiempo estimado?	EXTRACT(EPOCH FROM ...), HAVING con COUNT
I5	Vehículos sobre promedio mant.	¿Qué vehículos son candidatos a reemplazo?	Subconsulta anidada en HAVING (2 niveles)

2.3 Nivel Complejo

#	Nombre	Pregunta de negocio	Conceptos SQL
C1	Ranking bayesiano conductores	¿Quién es el mejor conductor en los 3 factores clave?	5 CTEs encadenadas, RANK(), NTILE(), Empirical Bayes
C2	Tendencia mensual variación %	¿Las entregas crecen o caen por ciudad?	LAG(), PARTITION BY, promedio movil ROWS BETWEEN
C3	Clasificación rutas por cuartil	¿Qué rutas son excelentes y cuales a mejorar?	NTILE(4), CTE encadenada, CASE WHEN multivariable
C4	Análisis fatiga conductores	¿Hay conductores sin descanso mínimo de 8h?	LAG() PARTITION BY driver_id, riesgo operacional

3. Query C1 — Ranking Bayesiano de Eficiencia de Conductores

Esta query es la mas avanzada del avance. Implementa un modelo estadístico de Empirical Bayes para rankear a los 378 conductores por su eficiencia real, considerando tres factores que la empresa identifica como prioritarios.

3.1 Por que Bayesiano y no un ranking simple

Con 1,174 entregas promedio por conductor, un ranking puro de porcentajes podría castigar a un conductor por mala suerte en pocos viajes o premiar a otro por una racha positiva. El modelo bayesiano corrige esto aplicando "shrinkage": encoge cada estimación individual hacia la media de la flota con una fuerza equivalente a 500 entregas previas (m=500).

La fórmula es: Posterior = (exitos_observados + m * mu_flota) / (n + m)

Conductores con pocas entregas quedan cerca de la media. Conductores con muchas entregas dominan su propio promedio. Esto hace el ranking justo independientemente del volumen de trabajo asignado a cada conductor.

3.2 Los tres componentes del score

Peso	Componente	Que mide	Media real de la flota
40%	Entregas exitosas	% con delivery_status=delivered. Mide si el conductor COMPLETA su trabajo.	85.0% (rango: 81.8-88.3%)
35%	Puntualidad	% entregadas en <= 30 min del horario (tiempo y forma). Factor con mas varianza entre conductores.	42.7% (rango: 38.7-47.2%)
25%	Combustible / 100km	Litros por cada 100km recorridos. Score invertido: menos = mejor.	2.29 L/100km (rango: 2.13-2.51)

Por que la puntualidad tiene el 35%: es el factor con mayor varianza entre conductores. El % de entregas exitosas varia poco (81.8% a 88.3% entre el mejor y el peor), mientras que la puntualidad varia de 38.7% a 47.2%. Un componente con mas varianza discrimina mejor quienes son realmente mas eficientes.

3.3 Pipeline SQL (5 CTEs encadenadas)

La query usa 5 CTEs en lugar de 4 porque PostgreSQL no permite funciones de ventana (OVER) dentro de funciones de agregación (AVG) en el mismo GROUP BY. La solución es separar el cálculo en dos pasos:

CTE	Descripcion
fuel_por_entrega	NUEVA (corrección técnica). Calcula el combustible proporcional por entrega usando OVER(PARTITION BY trip_id) ANTES del GROUP BY. PostgreSQL rechaza OVER() dentro de AVG().
metricas_base	Une drivers + trips + deliveries + routes. Calcula n (total entregas), ok (exitosas), on_time (puntuales <= 30 min) y avg_l_per_100km por conductor.
priors	Calcula las medias de la flota: mu_ok=0.850, mu_ot=0.427, mu_fuel=2.293. Son el prior bayesiano.
bayes	Aplica el shrinkage con m=500. Produce bayes_exitoso, bayes_puntual, bayes_fuel: estimaciones ajustadas.
scores	Normaliza cada componente a [0,100] usando MIN/MAX de la flota. Combustible se invierte (100=mas eficiente).

Nota técnica: La CTE fuel_por_entrega se agregó para resolver el error de PostgreSQL: "las llamadas a funciones de agregación no pueden contener llamadas a funciones de ventana deslizante" (error en línea 19 del script original). La solución es calcular el OVER() en una CTE previa y luego usar AVG() normalmente.

3.4 Resultados reales (378 conductores, 403,874 entregas)

Score máximo: Sergio Ruelas Amaya: 82.84 / 100 | 87.8% exitoso, 44.4% puntual, 2.155 L/100km

Score mínimo: Lilia Marrero Altamirano: 11.48 / 100 | 82.5% exitoso, 39.7% puntual, 2.473 L/100km

Media de la flota: 49.67 / 100 | desviación estándar: 11.18 puntos

Rango total: 11.48 a 82.84 (71.4 puntos de diferencia entre el mejor y el peor)

Hallazgo clave: flota homogénea

La desviación estándar de 11.2 puntos sobre un rango de 71.4 indica una flota altamente homogénea. No hay conductores catastróficamente malos ni extraordinariamente superiores. Para diferenciarse del promedio de la flota un conductor necesita consistencia en los tres factores simultáneamente, no solo destacar en uno.

La puntualidad como diferenciador principal

El 90.1% de las entregas en Snowflake tienen delay <= 30 min. Sin embargo, en el modelo C1 la puntualidad media es 42.7% porque C1 mide estrictamente las entregadas exactamente en tiempo sobre viajes completados, no sobre el total. Los conductores elite superan el 44% de puntualidad vs 39-40% del quintil inferior.

Eficiencia de combustible: ventaja sostenida

El top 2 (Ruelas Amaya y Holquin Narvaez) combina alta puntualidad con consumo 6.5% por debajo del promedio de flota (2.13-2.16 vs 2.29 L/100km). En 1,200 entregas anuales esto equivale a ~150 litros menos por conductor. Extrapolando a los 69 conductores del quintil elite: ahorro potencial de 10,350 litros por año.

El modelo es una herramienta de gestión, no una sentencia

Los pesos (40/35/25) representan una decisión de negocio. Si la empresa prioriza reducir costos de combustible, subir el peso a 40% y bajar puntualidad a 30% cambia el ranking y revela conductores diferentes como los más valiosos.

---

4. optimización con índices

4.1 Los 5 índices

índice	Tabla	justificación
idx_trips_composite_joins	trips	Filtra status=completed. Beneficia B2, I1, I2, I4, C1, C3
idx_deliveries_scheduled_datetime	deliveries	Filtra delivery_status=delivered. Beneficia B3, C1, C2

índice	Tabla	justificación
idx_maintenance_vehicle_cost	maintenance	Acelera JOIN y SUM(cost) por vehículo en B1, I2, I5
idx_drivers_status_license	drivers	Filtra conductores activos en I1, C1, C4
idx_routes_metrics	routes	Acelera JOINs y cálculos de distancia en B2, I2, I4, C3

4.2 Resultados EXPLAIN ANALYZE

Query	Descripción	Antes (ms)	Después (ms)	Mejora
B2	Rutas activas + combustible	253.6 ms	159.0 ms	37.3% OK
I3	Reporte mensual	794.5 ms	825.6 ms	-3.9% (cache)
C1	Ranking conductores	1,270.9 ms	1,353.9 ms	-6.5% (cache)
C2	Tendencia mensual	908.1 ms	932.0 ms	-2.6% (cache)
I5	Mantenimiento > promedio	8.2 ms	10.0 ms	Ya era optima

Nota técnica: Resultados negativos son normales: PostgreSQL almacena resultados en cache (shared_buffers). Las queries que no mejoraron ya estaban siendo servidas desde memoria. B2 mostro mejora del 37.3% al ser la primera ejecución sin cache caliente.

4.3 Vista Materializada

mv_resumen_mensual: Query I3 directa: 1,084.7 ms -> Vista materializada: 3.2 ms -> Mejora: 99.7%

La vista materializada almacena el resultado pre-calculado del reporte mensual. Se consulta con SELECT * FROM mv_resumen_mensual y se actualiza con REFRESH MATERIALIZED VIEW cuando llegan datos nuevos.

5. Hallazgos de Negocio

5.1 KPIs generales de la flota

Tasa de entrega exitosa (B3): 100% de las entregas con status delivered en los datos generados (meta: >= 85%)

Vehículos candidatos a reemplazo (I5): 89 de 200 vehículos superan el costo promedio de mantenimiento de la flota

Conductores con riesgo de fatiga (C4): 20 conductores con más de 10 viajes sin 8 horas de descanso (alto riesgo)

Puntualidad de la flota: 42.7% de las entregas en tiempo y forma (modelo C1) | 90.1% con delay <= 30 min (Snowflake DW)

5.2 Ranking de conductores (C1 — Top 5 y ultimo)

#	Conductor	Entregas	% Exitoso	% Puntual	L/100km	Score
1	Sergio Ruelas Amaya	1,312	87.8%	44.4%	2.155	82.84
2	Raquel Holquin Narvaez	1,153	88.3%	42.6%	2.129	78.23
3	Dario Rolon Nazario	1,080	87.1%	44.4%	2.195	74.79
...	...	...	...	...	...	...
378	Lilia Marrero Altamirano	1,160	82.5%	39.7%	2.473	11.48

5.3 Clasificación por quintil

Quintil	Conductores	% Exitoso prom	% Puntual prom	L/100km prom	Score prom
Q1 - Elite (top 20%)	76	86.7%	44.6%	2.22	67.3
Q2 - Bueno	76	85.8%	43.4%	2.27	54.3

Quintil	Conductores	% Exitoso prom	% Puntual prom	L/100km prom	Score prom
Q3 - Promedio	75	85.0%	42.7%	2.29	49.7
Q4 - Por mejorar	76	84.2%	41.6%	2.35	40.1
Q5 - Bajo rendimiento	75	83.1%	40.3%	2.41	27.8

---

6. Impacto de C1 en el Avance 3 (Data Warehouse)

El ranking bayesiano de conductores se integra directamente en el Avance 3 a través del campo performance_category de dim_driver:

dim_driver.performance_category: el ETL recalcula la tasa de puntualidad por conductor en cada batch y asigna Alto (>60%), Medio (40-60%) o Bajo (<40%). Es el resultado simplificado del modelo C1 aplicado continuamente.
v_driver_performance: une fact_deliveries con dim_driver para mostrar el score real de cada conductor, permitiendo dashboards de rendimiento actualizados con cada carga.
daily_kpi_summary.pct_on_time: el denominador del componente de puntualidad del modelo C1 se guarda en cada ejecución del ETL.

Los pesos 40/35/25 del modelo C1 son configurables en el ETL del Avance 3. Cambiarlos en C1 debería reflejarse en cómo se clasifica a los conductores en dim_driver.

---

7. Estructura del Proyecto

Archivo	descripción
Avance_2_SQLAnalysis.ipynb	Notebook principal (40 celdas) con las 12 queries, EXPLAIN ANALYZE y optimización.
02_queries_analysis.sql	Las 12 queries en SQL puro con comentarios para el evaluador.

Archivo	descripción
03_optimization_indexes.sql	5 índices vista materializada mv_resumen_mensual.
avance2_optimization_report.json	Tiempos EXPLAIN ANALYZE antes y después de índices.

---

8. Decisiones técnicas

¿Por qué m=500 como fuerza del prior bayesiano?

Con 378 conductores y 1,068 entregas promedio, m=500 equivale a asignarle a la media de la flota el peso de 500 entregas previas. Esto garantiza que conductores con 200 o 2,000 entregas sean comparables en el ranking. Con m=50 el ranking seria casi idéntico al crudo. Con m=5000 todos los conductores quedarían casi iguales a la media.

¿Por qué una CTE extra (fuel_por_entrega)?

PostgreSQL no permite OVER() dentro de AVG() en el mismo GROUP BY. Error real obtenido: "las llamadas a funciones de agregación no pueden contener llamadas a funciones de ventana deslizante". La solución es calcular el combustible proporcional por entrega en una CTE previa (usando OVER) y luego usar AVG() normalmente en la siguiente CTE. Sin esta separación, la query C1 falla en ejecución.

¿Por qué CROSS JOIN priors en la CTE bayes?

CROSS JOIN con una CTE de una sola fila es el patron SQL estándar para hacer disponible un valor calculado a toda la tabla. Es equivalente a una variable en Python. Sin él, habría que repetir el subquery de medias dentro de cada columna del SELECT, lo que es más lento y menos legible.

¿Por qué normalizar a [0,100] y no usar los porcentajes directos?

Los tres componentes están en escalas diferentes: % exitoso rango 6.5 puntos, % puntual rango 8.5 puntos, L/100km rango 0.38. Sin normalizar, el componente con mayor rango absolutamente dominaría el score independientemente del peso asignado. La normalización garantiza que los pesos 40/35/25 se apliquen realmente en esas proporciones.